

Module 2: Stacks and Queues.

- 1.1 Introduction, ADT of Stack, Operations on Stack, Array Implementation of Stack.
- 1.2 Applications of Stack - Well formedness of Parenthesis
- 1.3. Infix to Postfix Conversion.
- 1.4. Postfix Evaluation.
- 1.5. Recursion.
- 1.6. Introduction, ADT of Queue, Operations on Queue, Array Implementation of Queue.
- 1.7. Implementation of circular and double Ended Queue. Priority Queue, Applications of Queue

* Introduction - STACKS

Stack is a linear data structure in which items are added or removed only at one end, called top of the stack. Thus, there is no other way to add to delete elements.

Stack is based on Last-In-First-Out principle. i.e the data item that is inserted last into the stack is the first one to be removed from the stack.

Stack Representation in Memory

Stack appears as a group of elements stored at contiguous locations in memory. Each successive insert or delete operation adds or removes an item from the group. The top location of the stack or the point of addition or deletion is maintained by a pointer called top.

logical
representation
of
stack

top $\rightarrow$

87
51
62
29
35

* Operations on Stack.

There are two key operations performed on the stack - push and pop. Push operation refers to adding a new element to the stack whereas pop operation refers to reading or deleting an element from the stack.

1. Push operation

Algorithm

Step 1: Start

Step 2: IF $\text{top} = \text{MAX} - 1$

print "OVERFLOW" and exit.

else

go to step 3 [end of if]

Step 3: Set $\text{top} = \text{top} + 1$

Step 4: Set $\text{Stack}[\text{top}] = \text{element}$

Step 5: End.

The push operation involves the following subtasks.

1. Reviewing the elements to be inserted
2. Incrementing the stack pointer, top.
3. Storing reviewed element at new location of top

Visual representation of push operation.

		top →	68
top →	56		56
	35		35
	39		39
	42		42

push(68)

Element 68 is inserted at the top of the stack and top pointer points to new location

OVERFLOW condition

Stack is a ~~star~~ datatype having a fixed variable. If the stack is full and the user wants to add a new element, the stack cannot add a new element such a condition is called "STACK OVERFLOW".

top →	7	top →	7
	6		6
	5		5
	4		4
	3		3
	2		2
	1		1

push(8)
→

No space in stack, so "OVERFLOW"

2. pop operation

Algorithm

Step 1: Start

Step 2: IF $\text{top} = -1 / \text{NULL}$
print "UNDERFLOW" and exit
else

go to step 3 [End of if]

Step 3: Set $\text{element} = \text{stack}[\text{top}]$

Step 4: Set $\text{top} = \text{top} - 1$

Step 5: Stop.

The pop operation involves the following subtasks:

1. Retrieving or removing the element at the top of the stack.
2. Decrementing the stack pointer, top.

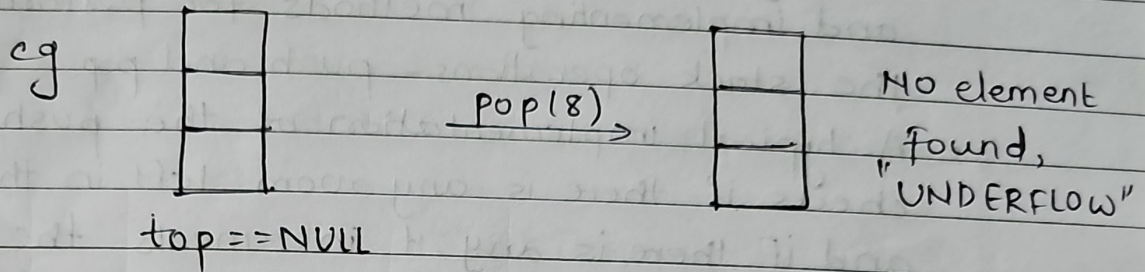
Visual representation of pop operation.

top →	23		
	46	top →	46
	9		9
	12		12
	36		36

pop(23)

UNDERFLOW CONDITION

If we want to perform pop operation on an empty stack, the condition is referred to as underflow condition.



* ADT of Stack

- ADT means what operations are to be performed on the data, not the implementation of the operations.
- In Stack ADT, implementation, instead of data being stored at each node, the pointer of the data is stored.
- The program allocates memory to data and the address / pointer to the ~~stack~~ data is passed to the stack. The head node and the data nodes are encapsulated in ADT. The calling functions can only see the pointer to the stack.
- The stack head structure also contains a pointer to top and count of number of entries currently in stack.
- Operations on Stack ADT.
 1. push() - inserts element at one end of the stack.
 2. pop() - deletes or returns element from the top of stack.
 3. peek() - returns the element at the top of the stack without removing it.
 4. size() - returns the number of elements in the stack.
 5. isEmpty() - returns true if stack is empty, else returns false.
 6. isFull() - returns true if stack is full, else returns false.

* Stack implementation

1. Stack implementation involves choosing the data storage mechanism for storing stack elements and implementing methods for performing the stack operations - push and pop.

2. A typical implementation of the push operation checks if there is any room left in the stack, and if there is any, it increments the stack counter by one and inserts the received item at the top of the stack.

Similarly, the implementation of the pop operation checks whether or not stack is empty, if it is not, it removes the top element of the stack and decrements the stack counter by one.

3. We can implement stacks by using arrays or linked lists.

(Stack implementation using linked lists in Linked list Module)

Array implementation of stack

↳ It involves allocation of fixed size array in the memory.

↳ Both stack operations (push & pop) are made on this array with constant check that array does not go out of bounds.

Other operations on stack

peek operation

returns the value at the topmost position.

Algorithm:

Step 1: Start

Step 2: if $top = NULL$

 print "stack is empty" and exit
 else

 return $stack[top]$

Step 3: End.

Display code

```
void display()
```

```
{
```

```
    for ( $i = top; i \geq 0; i--$ )
```

```
    {
```

```
        printf("%d",  $stack[i]$ );
```

```
    }
```

```
    if ( $top == -1$ )
```

```
    {
```

```
        printf("Stack is empty");
```

```
    }
```

```
}
```

Implementation Can be implemented using array
Datatype It is a ADT

Cannot be implemented using stack
Array is a datatype

* Stacks vs Arrays.

Parameters	Stack	Arrays
Definition	It is a linear DS represented by a sequential collection of elements in a fixed order	It is a collection of related data values called elements each identified by an index
Principle	Based on LIFO principle	Elements belong to index
Access	In Stack, only topmost element can be accessed	In array, elements can be accessed using index
Storage	Dynamic size	* Fixed size
Operation	Insertion & Deletion can be done at one end	Insertion and deletion via array index

* Application of Stacks

8 marks

1. Well formedness of parenthesis
2. Infix to postfix Conversion
3. Postfix Evaluation
4. Recursion.
5. Reversing a list.
6. Parenthesis checker.

1. Infix to Postfix Expression

Expressions can be of three types-

1. Infix - $a + b$
2. Postfix - $ab +$
3. Prefix - $+ab$.

Prefix & Postfix expression does not contain parenthesis.

Algorithm.

Step 1: Start

Step 2: Repeat the following until each element of the ^{infix}~~stack~~ expression is scanned.

If a "(" is encountered, PUSH TO STACK

If a ")" is encountered, POP repeatedly from the stack and PUSH to postfix until "(" is encountered. Discard "(" and do not add to postfix expression.

If an operand is encountered, PUSH to postfix expression.

If an operator is encountered,

check If the operator in the stack has same / higher precedence

POP operator in the stack and PUSH to postfix expression. PUSH the encountered operator to the stack.

else

PUSH encountered operator in the stack.

[End of if]

[End of if]

Step 3: POP all the remaining elements in the stack repeatedly until stack is empty and PUSH to postfix expression.

Step 4: End.

Q1. Convert infix expression to postfix expression using stack.

$$A - (B / C + (D \% E * F) / G) * H.$$

Ans.

Input	Stack	Postfix Expression
A		A
-	- $\leftarrow$ top	A
(	- $\leftarrow$ top	A
B	- $\leftarrow$ top	AB
/	- $\leftarrow$ top	AB
C	- $\leftarrow$ top	ABC
+	- $\leftarrow$ top	ABC/
(	- $\leftarrow$ top	ABC/
D	- $\leftarrow$ top	ABC/D
%	- $\leftarrow$ top	ABC/D
E	- $\leftarrow$ top	ABC/DE
*	- $\leftarrow$ top	ABC/DE%
F	- $\leftarrow$ top	ABC/DE%.F
)	- $\leftarrow$ top	ABC/DE%.F*
/	- $\leftarrow$ top	ABC/DE%.F*
G	- $\leftarrow$ top	ABC/DE%.F*G
)	- $\leftarrow$ top	ABC/DE%.F*G/
*	- $\leftarrow$ top	ABC/DE%.F*G/+
H		ABC/DE%.F*G/++H*-

Q2 Convert the infix expression to postfix expression.
 $A * (B * C + D * E) + F$

Ans.

Input	Stack	Postfix Expression
A		A
*	* $\leftarrow$ top	A
(	* ($\leftarrow$ top	A
B	* ($\leftarrow$ top	AB
*	* (* $\leftarrow$ top	AB
C	* (* $\leftarrow$ top	ABC
+	* (+ $\leftarrow$ top	ABC*
D	* (+ $\leftarrow$ top	ABC*D
*	* (+* $\leftarrow$ top	ABC*D
E	* (+* $\leftarrow$ top	ABC*DE
)	* $\leftarrow$ top	ABC*DE*+
+	+ $\leftarrow$ top	ABC*DE*+*
F	+ $\leftarrow$ top	ABC*DE*+*F
		ABC*DE*+*F+

2. Postfix Evaluation

Evaluating a given postfix expression

Step 1: Start

Step 2: Add ")" to the given postfix expression

Step 3: Scan every character of postfix expression and repeat Step 4 until ")" is encountered.

Step 4: If an operand is encountered, push it to stack

If an operator $\odot$ is encountered, then POP two elements from the stack as A and B, where A is the topmost element and B is the second topmost element.

Evaluate $B \odot A$

PUSH the result to the Stack.

[end of if]

Step 5: Set result = topmost element

Step 6: End.

Q1 Evaluate the following postfix expression.

$$23-4+567*+*$$

Ans.

$$PE: 23-4+567*+*)$$

Input	Stack	Postfix evaluation
2	$2 \leftarrow \text{top}$	
3	$23 \leftarrow \text{top}$	
-	$-1 \leftarrow \text{top}$	$2-3 = -1$
4	$-14 \leftarrow \text{top}$	
+	$3 \leftarrow \text{top}$	$-1+4 = 3$
5	$35 \leftarrow \text{top}$	
6	$356 \leftarrow \text{top}$	
7	$3567 \leftarrow \text{top}$	
*	$3542 \leftarrow \text{top}$	$6*7 = 42$
+	$347 \leftarrow \text{top}$	$5+42 = 47$
*	$141 \leftarrow \text{top}$	$3*47 = 141$

Result = 141.

3. Wellformedness of Parenthesis

Parenthesis - $()$, $\{\}$, $[\]$

Balanced expression means that there are opening and closing brackets present. Well formedness of Parenthesis means that the expression is balanced i.e. for each opening bracket there is a closing bracket.

Algorithm

Step 1: Start

Step 2: Traverse each character in the string expression.

Step 3: Repeat the following until end of the string is reached

if " $($ ", " $\{$ ", " $[$ " is encountered,

PUSH to stack,

if " $)$ ", " $\}$ ", " $]$ ",

check if the encountered parenthesis matches with the ^{opening} ~~open~~ closing parenthesis on the top of the stack.

POP the parenthesis on the top.
else unbalanced, exit
[end of if]

Step 4: If stack is empty, then expression is balanced

else

unbalanced

[end of if]

Step 5: Exit

Q1 Check the well formedness of the expression $[a+b-(x+y)*\{c+d\}]$

I / P	Stack
[	[$\leftarrow$ top
a	[$\leftarrow$ top
+	[$\leftarrow$ top
b	[$\leftarrow$ top
-	[$\leftarrow$ top
(	[($\leftarrow$ top
x	[($\leftarrow$ top
+	[($\leftarrow$ top
y	[($\leftarrow$ top
)	[$\leftarrow$ top
*	[$\leftarrow$ top
{	[{ $\leftarrow$ top
c	[{ $\leftarrow$ top
+	[{ $\leftarrow$ top
d	[{ $\leftarrow$ top
}	[$\leftarrow$ top
]	

Expression is balanced

* 4 Recursion.

Calling of a function inside a function.

eg. Factorial: `#include <stdio.h>`

```
int f(int x)
```

```
{
```

```
    if (x == 0 || x == 1)
```

```
        return 1;
```

```
    if & else
```

```
        return x * f(x-1);
```

```
}
```

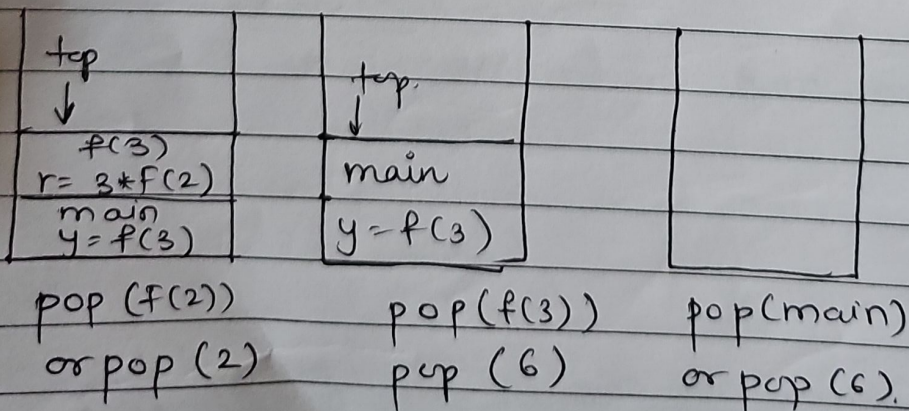
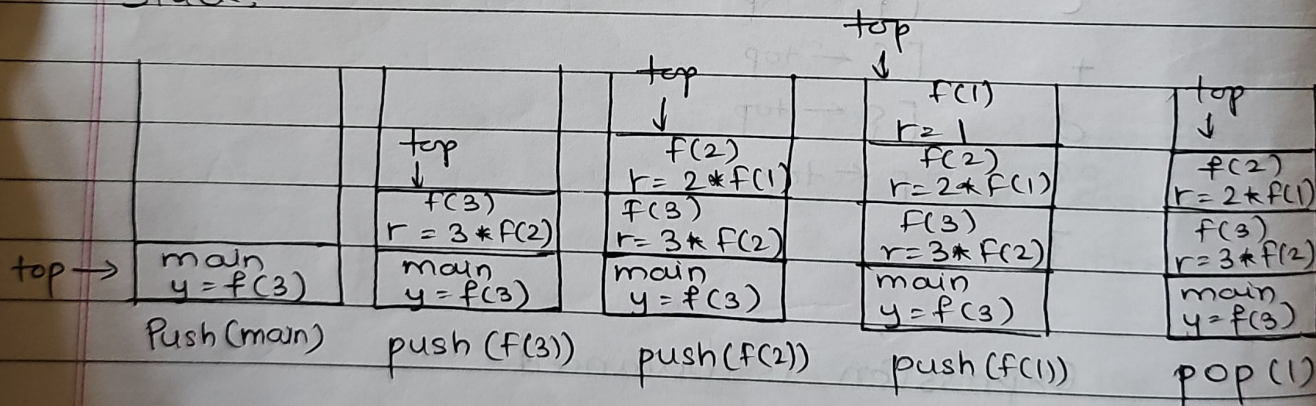
```
void main()
```

```
{
```

```
    int y = f(3);
```

```
}
```

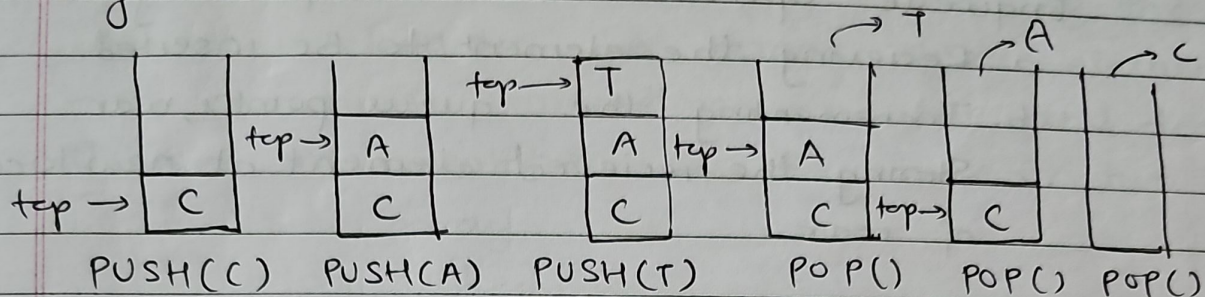
Stack.



∴ Result = 6.

* Reversing a string

Eg. Reverse a word "CAT"



∴ Result: TAC

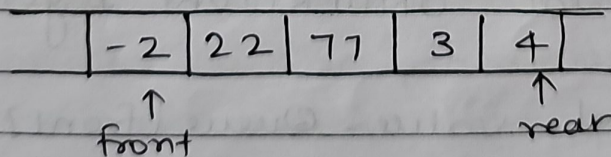
* Queues

Queue is a linear data structure in which elements are inserted at one end called Rear and deleted from another end called front. Queues are based on the principle first-in-first-out (FIFO). Queues are used in CPU scheduling, resource sharing etc.

Operations performed on queue

1. enqueue - Insert elements in queue
2. dequeue - Delete elements from queue

Logical Representation of Queue



There are two pointers in queue - front and rear. Insertion operation in a queue occurs at one end called rear and deletion operation in a queue occurs at other end called front. Overflow and underflow conditions occur in queue as well.

Operations on queue

1. Enqueue

Enqueue operation has the following subtasks

- Receiving the element to be inserted
- Incrementing the queue pointer, rear
- Storing the received element at new location of rear.

Algorithm for enqueue

Step 1: Start

Step 2: If $\text{rear} = \text{MAX} - 1$

print OVERFLOW and go to step 4.

~~Step~~ Else if $(\text{front} = -1 \ \& \ \text{rear} = -1)$

set $\text{front} = \text{rear} = 0$

else

$\text{rear} = \text{rear} + 1$ [end of if]

Step 3: $\text{queue}[\text{rear}] = \text{Number}$

Step 4: Exit

2. Dequeue

Algorithm for dequeue

Step 1: Start

Step 2: if $\text{front} = -1$ OR $\text{front} > \text{rear}$

print "UNDERFLOW" & go to step 3.

~~Step~~ else

set $\text{value} = \text{Queue}[\text{front}]$

Set $\text{front} = \text{front} + 1$

[end of if]

Step 3: Exit

* Queue Implementation.

Queue implementation involves choosing the data storage mechanism for storing queue elements and performing operations, enqueue and dequeue. Queue implementation can be done in two ways.

1. Using array
2. Using linked list
(In next module)

Array implementation of queue

The array implementation of queue involves allocation of fixed size of array in the memory. Both the queue operations are done with a constant check that the queue does not go out of bounds.

* Queue as ADT

ADT refers to abstract data type i.e. the user does not need to know the implementation of data but the operations performed on the data.

In Queue ADT implementation, it follows the basic design of Stack ADT. Each node contains a void pointer to the data and link pointer to the next element in the queue. The program's responsibility is to allocate memory for storing data.

Operations ^{on} Queue ADT.

1. enqueue()
2. dequeue()
3. isFull()
4. isEmpty()

* Types of Queues

1. Simple or Linear Queue
2. Circular Queue
3. Priority Queue
4. Double Ended Queue or de-queue

1 Simple or Linear Queue

Linear queue follows FIFO principle i.e. first in first out. Insertion occurs at the rear end and Deletion occurs at the front end

Disadvantages

- a. Insertion and Deletion occur only at the ends.
- b. Operations such as insertion and deletion of elements in the middle is time consuming.
- c. In linear queue, a new element can only be inserted when existing elements are deleted from the queue.
- d. Memory space is wasted.

2 Circular queue

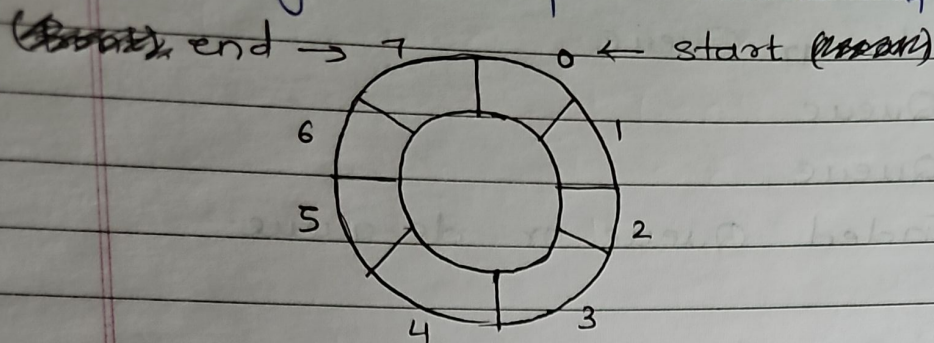
A circular queue is a queue whose rear and front ends are logically connected to each other.

That means, the start location comes after the end location. IF we continue to add elements

in a circular queue, ~~the~~ ~~are~~ till the end location, then the new element would be added to the beginning of the queue. Circular queues

remove one of the major disadvantages of linear or array implemented queue that is inefficient utilization of memory space. If a queue is filled till the end location, then the start location will be searched for an empty space & if an empty space is found, the new element will be added there.

Logical Representation of Circular queue



Operations on circular queue

↳ 1 enqueue - inserts elements in queue

Step 1: Start

Step 2: IF $\text{front} = 0$ and $\text{rear} = \text{Max} - 1$ OR
 $\text{rear} = \text{front} = \text{rear} + 1$

print "OVERFLOW" and go to
step. 4.

~~Step 3~~ else if $\text{front} = -1$ and $\text{rear} = -1$
set $\text{front} = \text{rear} = 0$

else if $\text{rear} = \text{Max} - 1$ and $\text{front} \neq 0$
set $\text{rear} = 0$

else

set $\text{rear} = \text{rear} + 1$. [end of if]

Step 3: Set $\text{Queue}[\text{rear}] = \text{value}$

Step 4: Exit

↳ 2. dequeue - deletes elements in queue

Step 1: Start

Step 2: if $\text{front} = -1$

print "UNDERFLOW"

go to step 3

else if $\text{front} = \text{rear}$

set $\text{front} = \text{rear} = -1$

else

if $\text{front} = \text{max} - 1$

set $\text{front} = 0$

else

$\text{rear} = \text{front} = \text{front} + 1$

[end of if]

[end of if]

Step 3: Exit

Applications of Queues

↳ traffic lights.

3. Double-Ended Queue or DeQueue

A double-ended Queue or De-Queue is a special type of queue which allows insertion and deletion of elements at both the ends i.e. rear & front. but not at the middle.

Based on the type of restrictions, it is classified as:

1. Input-restricted de-queue: Allows deletion at both ends but insertion at one end.
2. Output-restricted de-queue: Allows insertion at both ends but deletion at one end.

Operations on dequeue

1. Insertion at rear
2. Insertion at front
3. Deletion at rear
4. Deletion at front

1. Insertion at rear.

Algorithm:

Step 1: Start

Step 2: if (front == 0 && rear == MAX-1)

|| (front == rear)

Print "overflow" and goto step 5.
else if (~~front == MAX-1~~) (rear == -1)

set front = rear = rear + 1

queue[rear] = element

return;

[end of if]

Step 3: rear = rear + 1

Step 4: queue[rear] = element

Step 5: exit

Algorithm

1. Insertion at rear

Step 1: Start

Step 2: if (rear == MAX-1 && front == 0) ~~front~~
printf ("OVERFLOW")

go to step. 4

else if (rear == -1)

front = rear = rear + 1

queue[rear] = element

else if (rear == MAX-1) && front != 0

rear = 0

else

rear = rear + 1 [end of if]

Step 3: dequeue [rear] = element.

Step 4: Exit.

2. Insertion at front

Step 1: Start.

Step 2: if (front == 0 and rear == MAX-1) ||

front = rear + 1,

print OVERFLOW, and goto step 4

else if front == -1

front = rear = 0.

else if front == 0.

front = MAX-1

else

front = front - 1. [end of if]

Step 3: dequeue [front] = element

Step 4: exit.

3. Deletion at rear()

Step 1: Start

Step 2: if front == -1

print "UNDERFLOW" and go to step 3.

else if [front == rear]

front = rear = -1.

else if (rear == 0)

rear = MAX-1

else

rear = rear - 1

[end of if]

Step 3: Exit.

4. Deletion at front.

Step 1: Start.

Step 2: if $\text{front} == -1$

print UNDERFLOW and goto step 3

else if $(\text{front} == \text{rear})$

$\text{front} = \text{rear} = -1$

else if $(\text{front} == \text{MAX} - 1)$

$\text{front} = 0$.

else

$\text{front} = \text{front} + 1$;

[end of if]

Step 3: Exit

* Priority Queue

A priority queue is a type of queue in which each element is assigned with certain priority such that the order of ^{processing or} deletion of elements is decided by their associated priorities.

Rules for order of processing.

1. An element with higher priority is processed before an element with lower priority.
2. In case both elements have same priority they are processed on the basis of first come first served.

Applications of Priority Queue

1. Used in Dijkstra's shortest path algorithm.
2. Used in Prim's Algorithm.
3. Used in data compression techniques like Huffman code
4. Used in heap sort.
5. Used in OS for priority scheduling, and interrupt handling